

Pygame word scramble validation bug

94 lines

PY

89 lines

MD

28 lines

PY

I'm working on a Pygame Word Scramble game ,

Task 1: typing the correct unscrambled word is rejected as wrong, while typing the scrambled letters is accepted. Find the bug in `submit_guess` and fix it so guesses are validated against the secret word. Show only the changed lines and explain the cause in one or two sentences. Do not change any other behavior, i will need individual git commit per task so give that accordingly

12 hours ago

Bug location: `submit_guess` in `game_engine.py`

Change:

diff

```
-         is_correct = (guess == self.scrambled_word)
+         is_correct = (guess == self.secret_word)
```

Cause: The guess was being compared against `self.scrambled_word` (the jumbled display text) instead of `self.secret_word` (the actual answer), so the check was inverted from what the game intends. Nothing else needs to change; the rest of the method (`score`, `feedback`, `next_round()`, clearing the box) already behaves as the README describes.

Commit for Task 1:

bash

```
git add game/game_engine.py
git commit -m "Fix guess validation: compare against secret_word, not
scrambled_word"
```

Send Task 2 whenever you're ready and I'll give its commit the same way.

12 hours ago

Task 2: add a progressive hint system to game_engine.py. Requirements:

- A HINT button next to the SUBMIT button (same style).
- Each click reveals one more letter of the secret word in its correct position, shown as a line like "P _ _ _ E" above the input box.
- Each hint reduces the points earned for that round by 0.25, with a floor of 0. Show the score cleanly (e.g. 2.5, not 2.50).
- Hints and revealed letters reset at the start of each round.
- Don't reveal the entire word; stop giving hints when one letter is left.

Keep all existing behavior. Show only the new or changed code and say where each piece goes plus give me the git commit change

Working out hint reveals and scoring for each round.

All changes are in game/game_engine.py.

1. In `__init__`, next to the existing submit_btn and font lines:

python

```
self.hint_btn = pygame.Rect(width // 2 + 150, 210, 95, 46)
```

python

```
self.font_hint = pygame.font.SysFont(None, 32)
```

2. In `next_round`, add one line (resets hints each round):

python

```
def next_round(self):
    self.secret_word = random.choice(self.words)
    self.scrambled_word = self.scramble_string(self.secret_word)
    self.revealed = set()          # <-- new
    self.input_box.clear()
```

3. New method, placed right after next_round:

python

```
def use_hint(self):
    hidden = [i for i in range(len(self.secret_word)) if i not in
self.revealed]
    if len(hidden) <= 1:
        self.feedback_msg = "No more hints available!"
        self.feedback_color = (240, 170, 50)
    return
```

```

self.revealed.add(random.choice(hidden))
self.feedback_msg = "Hint used: -0.25 points this round."
self.feedback_color = (240, 170, 50)

```

4. In `submit_guess`, replace the score line in the correct branch:

diff

```

-         self.score += 1
+         self.score += max(0, 1 - 0.25 * len(self.revealed))

```

5. In `handle_event`, extend the mouse branch:

diff

```

        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if self.submit_btn.collidepoint(event.pos):
                self.submit_guess()
+            elif self.hint_btn.collidepoint(event.pos):
+                self.use_hint()
+                self.input_box.active = True

```

The last line keeps the text box focused after clicking HINT. Without it, `TextBox` deactivates on any click outside its `rect`, and the player would have to click the box again before typing.

6. In `render`, make these three edits:

Score display, with `:g` so 2.5 shows as 2.5 and 3.0 as 3:

diff

```

-         score_surf = self.font_msg.render(f"Score: {self.score}", True, (255,
220, 80))
+         score_surf = self.font_msg.render(f"Score: {self.score:g}", True,
(255, 220, 80))

```

Hint line, added after the `scramble_surf` blit and before `self.input_box.render(screen)`:

python

```

        hint_line = " ".join(
            ch if i in self.revealed else "_" for i, ch in
enumerate(self.secret_word)
        )
        hint_surf = self.font_hint.render(hint_line, True, (255, 220, 80))
        screen.blit(hint_surf, (self.width // 2 - hint_surf.get_width() // 2,
178))

```

HINT button, added after the SUBMIT button drawing code:

python

```
pygame.draw.rect(screen, (50, 150, 85), self.hint_btn, border_radius=6)
pygame.draw.rect(screen, (220, 220, 220), self.hint_btn, width=2,
border_radius=6)
hint_text = self.font_btn.render("HINT", True, (255, 255, 255))
screen.blit(hint_text, (self.hint_btn.centerx - hint_text.get_width()
// 2, self.hint_btn.centery - hint_text.get_height() // 2))
```

Notes

Hints reveal random positions, and `len(self.revealed)` doubles as the hint count, so no extra counter is needed.

Hints stop once one letter remains hidden, and clicking HINT then shows “No more hints available!” with no penalty.

A round’s points are $\max(0, 1 - 0.25 \times \text{hints})$, so 4 or more hints earns 0.

I haven’t seen `main.py`, so I don’t know your window width. The HINT button’s right edge is at `width // 2 + 245`, so the window needs to be at least ~490px wide. If it’s narrower, shift the buttons left.

Commit for Task 2:

bash

```
git add game/game_engine.py
git commit -m "Add progressive hint system with 0.25 point penalty per hint"
```

Send Task 3 when ready.

HINT button, added after the SUBMIT button drawing code:---which exact code line?

Locating where to insert new button code.

Paste it right after the `screen.blit(btn_text, ...)` line, and before the `feedback_surf` line. In render, it looks like this:

python

```
pygame.draw.rect(screen, (50, 150, 85), self.submit_btn,
border_radius=6)
pygame.draw.rect(screen, (220, 220, 220), self.submit_btn, width=2,
border_radius=6)
btn_text = self.font_btn.render("SUBMIT", True, (255, 255, 255))
screen.blit(btn_text, (self.submit_btn.centerx - btn_text.get_width()
```

```
// 2, self.submit_btn.centery - btn_text.get_height() // 2)) # <-- the HINT
block goes right after this line

        pygame.draw.rect(screen, (50, 150, 85), self.hint_btn, border_radius=6)
        pygame.draw.rect(screen, (220, 220, 220), self.hint_btn, width=2,
border_radius=6)
        hint_text = self.font_btn.render("HINT", True, (255, 255, 255))
        screen.blit(hint_text, (self.hint_btn.centerx - hint_text.get_width()
// 2, self.hint_btn.centery - hint_text.get_height() // 2))

        feedback_surf = self.font_msg.render(self.feedback_msg, True,
self.feedback_color) # <-- unchanged, stays below
```

Task 3: add a 30-second countdown timer per round. Requirements:

- A visible timer bar below the feedback message that shrinks as time passes and changes color from green to orange to red.
- Use `pygame.time.get_ticks()` rather than frame counting.
- When time runs out, show "TIME'S UP! The word was 'X'" in the feedback area and start the next round automatically, with no point awarded.
- The timer resets every round.

Keep the hint system working. Show only new or changed code, with locations.

Building a round timer for the game logic.

All changes are in `game/game_engine.py`.

1. In `__init__`, next to the other settings (before `self.next_round()` is called):

python

```
self.round_duration = 30000 # ms
```

2. In `next_round`, add one line so the timer resets every round (including after a correct answer or a timeout):

python

```
def next_round(self):
    self.secret_word = random.choice(self.words)
    self.scrambled_word = self.scramble_string(self.secret_word)
    self.revealed = set()
    self.round_start = pygame.time.get_ticks() # <-- new
    self.input_box.clear()
```

3. New method, placed right after next_round (or after use_hint):

python

```
def time_left_ms(self):
    elapsed = pygame.time.get_ticks() - self.round_start
    return max(0, self.round_duration - elapsed)
```

4. Replace the empty update method:

diff

```
def update(self):
-     pass
+     if self.time_left_ms() <= 0:
+         self.feedback_msg = f"TIME'S UP! The word was
'{self.secret_word}'"
+         self.feedback_color = (240, 80, 80)
+         self.next_round()
```

The message is built from `self.secret_word` before `next_round()` picks the new word, so it shows the word that just expired. No score is added, and `next_round()` also resets the hints and clears the input box.

5. In render, add this at the very end, after the `screen.blit(feedback_surf, ...)` line:

python

```
bar_w, bar_h = 300, 14
bar_x = self.width // 2 - bar_w // 2
bar_y = 320
frac = self.time_left_ms() / self.round_duration

if frac > 0.5:
    bar_color = (80, 230, 110)    # green
elif frac > 0.25:
    bar_color = (240, 170, 50)    # orange
else:
    bar_color = (240, 80, 80)      # red

pygame.draw.rect(screen, (50, 55, 65), (bar_x, bar_y, bar_w, bar_h),
border_radius=6)
pygame.draw.rect(screen, bar_color, (bar_x, bar_y, int(bar_w * frac),
bar_h), border_radius=6)
```

Notes

The bar is green above 15s, orange from 15s to 7.5s, and red below 7.5s.

Hints still work as before. They live in `self.revealed`, which resets with each round.

I haven't seen `main.py`, so two things to check. First, the main loop must call `game.update()` every frame, since the timeout check runs there. Second, the window height must be at least ~345px so the bar at `y=320` is visible. If it's shorter, lower `bar_y`.

Commit for Task 3:

bash

```
git add game/game_engine.py
git commit -m "Add 30-second per-round countdown timer with color-changing bar"
```

Send Task 4 when ready.

Task 4: replace the static scrambled text with interactive letter tiles.

Requirements:

- Each letter is drawn as a rounded tile, centered horizontally above the input box.
- Click one tile to select it (highlight it), then click another to swap them. Clicking the selected tile again deselects it.
- Tiles reset to the new scramble each round.
- Tile rearranging is just for experimenting. Guesses are still typed into the text box.

Keep everything else (hints, timer, scoring) working. Show only new or changed code, with locations.

Designing tile layout and swap-based selection for letter tiles.

All changes are in `game/game_engine.py`.

1. In `__init__`, next to the other fonts and settings (before `self.next_round()` is called):

python

```
self.font_tile = pygame.font.SysFont(None, 40)
self.tile_w, self.tile_h, self.tile_gap, self.tile_y = 42, 52, 8, 108
```

2. In `next_round`, add two lines so the tiles reset to the new scramble every round (including after a timeout):

python

```
def next_round(self):
    self.secret_word = random.choice(self.words)
```

```
self.scrambled_word = self.scramble_string(self.secret_word)
self.tiles = list(self.scrambled_word)    # <-- new
self.selected_tile = None                 # <-- new
self.revealed = set()
self.round_start = pygame.time.get_ticks()
self.input_box.clear()
```

3. Two new methods, placed right after `use_hint` (or anywhere among the other methods):

python

```
def tile_rects(self):
    n = len(self.tiles)
    total = n * self.tile_w + (n - 1) * self.tile_gap
    x0 = self.width // 2 - total // 2
    return [
        pygame.Rect(x0 + i * (self.tile_w + self.tile_gap), self.tile_y,
self.tile_w, self.tile_h)
        for i in range(n)
    ]

def handle_tile_click(self, pos):
    for i, rect in enumerate(self.tile_rects()):
        if rect.collidepoint(pos):
            if self.selected_tile is None:
                self.selected_tile = i
            elif self.selected_tile == i:
                self.selected_tile = None
            else:
                j = self.selected_tile
                self.tiles[i], self.tiles[j] = self.tiles[j], self.tiles[i]
                self.selected_tile = None
    return True

return False
```

4. In `handle_event`, extend the mouse branch with one more `elif`:

diff

```
elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
    if self.submit_btn.collidepoint(event.pos):
        self.submit_guess()
    elif self.hint_btn.collidepoint(event.pos):
```

```

        self.use_hint()
        self.input_box.active = True
+       elif self.handle_tile_click(event.pos):
+           self.input_box.active = True

```

As with HINT, this keeps the text box focused so the player can keep typing after clicking a tile.

5. In render, delete these three lines:

diff

```

-         spaced_letters = " ".join(self.scrambled_word)
-         scramble_surf = self.font_word.render(spaced_letters, True, (100, 200,
255))
-         screen.blit(scramble_surf, (self.width // 2 -
scramble_surf.get_width() // 2, 130))

```

and put this in their place (before the hint line and `self.input_box.render(screen)`):

python

```

        for i, (rect, ch) in enumerate(zip(self.tile_rects(), self.tiles)):
            selected = (i == self.selected_tile)
            fill = (100, 200, 255) if selected else (45, 52, 66)
            border = (255, 220, 80) if selected else (100, 200, 255)
            text_color = (26, 30, 38) if selected else (100, 200, 255)
            pygame.draw.rect(screen, fill, rect, border_radius=8)
            pygame.draw.rect(screen, border, rect, width=3, border_radius=8)
            letter = self.font_tile.render(ch, True, text_color)
            screen.blit(letter, (rect.centerx - letter.get_width() // 2,
rect.centery - letter.get_height() // 2))

```

Notes

The tile row spans `y=108` to `160`, so it sits between the score (`y=70`) and the hint line (`y=178`), directly above the input box.

The longest word (ALGORITHM, 9 letters) makes the row 442px wide, which fits within the ~490px minimum from Task 2.

Guessing still only reads `self.input_box.text`, so rearranging tiles never affects scoring, hints, or the timer.

`self.font_word` and `self.scrambled_word` are now unused for display, but I left them in place to avoid changing anything else.

A selection stays active if you click empty space, and it clears when the round changes.

Commit for Task 4:

bash

```
git add game/game_engine.py
git commit -m "Replace static scramble text with clickable swap-able letter
tiles"
```

12 hours ago